

CAT 3

The state is key

Alejandro Pérez Bueno

Apr 29, 2026

Table of Contents

Exercise 1	2
Exercise 2	2
Exercise 3	2
Component Example	2
Exercise 3	4
a)	4
b)	4
c)	4
Project	5
References	7

Exercise 1

Vue components follow a lifecycle that includes initialization steps such as data observation setup, template compilation, DOM mounting and DOM updates on data changes ([Vue.js 2024b](#)). The framework runs lifecycle hooks during this process. These hooks let developers add code at set points. Vue hooks integrate with its reactivity system, though they resemble native Web Components ([Azaustre Rodriguez 2024](#)).

Here are three lifecycle stages:

The **created** hook runs after instance setup. The reactivity system, data, computed properties and methods work at this point. The template remains uncompiled and no DOM attachment occurs. Use it for async tasks like API calls ([Vue.js 2024b](#)).

Once initial rendering finishes and nodes are in the document, the **mounted** hook triggers. Code needing direct DOM interaction belongs here such as tracking elements or setting up libraries ([Vue.js 2024b](#)).

The **unmounted** hook (called **destroyed** in Vue 2) executes on component removal from the DOM. It clears resources such as timers, subscriptions and event listeners to avoid leaks ([Vue.js 2024b](#)).

Exercise 2

Mounting the `CounterDisplay` component executes the `mounted()` lifecycle hook following DOM insertion. The `this.count++` statement then tries to modify the inherited `count` property. Vue responds by generating a console warning about prop. mutation. The local instance may show the new value, but the framework restricts this modification from altering the source.

Because of this the parent component observes no changes. Vue uses a one-way data flow model where properties move exclusively from parent to child ([Vue.js 2024c](#)). This structure allows for predictable application state. Child components treat received props as read-only and lack permission to override parent ownership. Direct prop modification violates this design, as components must emit events to request updates rather than changing values internally. The parent receives the event, executes the mutation, and passes the updated value. The `mounted()` logic bypasses this required communication cycle, which triggers the warning and preserves the parent data ([Vue.js 2024c](#)).

Exercise 3

This `<Teleport>` component renders content outside its parent in the DOM tree. The internal Vue context remains unchanged, including state, props and reactivity ([LogRocket Blog 2020](#)).

Component Example

```
<template>
  <Teleport to="body">
    <div v-if="isOpen" class="modal-overlay">
```

```
<div class="modal-content">
  <h2>Important Notification</h2>
  <p>This modal content is rendered outside its parent component.</p>
  <button @click="isOpen = false">Close Modal</button>
</div>
</div>
</Teleport>
</template>

<script>
import { ref } from 'vue';

export default {
  setup() {
    const isOpen = ref(false);

    const open = () => {
      isOpen.value = true;
    };

    return {
      isOpen,
      open,
    };
  },
};
</script>
```

This component addresses issues from component nesting, because elements like modals require position above other content. They must avoid parent CSS constraints like `overflow: hidden`. Without `<Teleport>`, state synchronization with global elements becomes complex ([LogRocket Blog 2020](#)).

Content avoids inheritance of parent styles or clipping at boundaries. Rendering occurs at the root or `<body>`, which provides a new context.

The component stays nested for data and logic and rendered output appears globally, which separates logical structure from visual placement.

Exercise 3

a)

`sumA` acts as a computed property that derives its value from `items`. `total` serves as a reactive reference updated through a watch effect.

`sumA` follows a declarative approach. Vue caches its result, recomputing only on changes to `items`. Access uses `sumA.value` ([Vue.js 2024a](#)).

`total` follows an imperative approach. A watch effect calculates and assigns the sum to `total` on `items` changes. `total` remains mutable as a ref.

`sumA` suits derived values from reactive state. `total` suits values updated by watching state changes.

b)

Use Option A for deriving state from reactive data like `items`, for read-heavy use with infrequent changes or for synchronous calculations.

Use Option B for side effects such as DOM changes, API requests, for logging on state changes or for updating separate reactive values with async logic.

c)

Feature	Option A	Option B
Performance	Caches value; recomputes on dependency changes only.	Runs on every source change; no caching.
Clarity	Shows value derivation from data.	Fits effects; less clear for simple derivations.
Flexibility	Synchronous returns only.	Handles async operations.
Mutability	Read-only by default.	Mutable ref.

Given the following access pattern:

- Initial Load: Both `sumA` and `total` are calculated.
- Access `sumA` 5 times: `sumA.value` returns the cached value 5 times. No calculation is performed after the first read.
- Access `total` 5 times: `total.value` reads the stored value from the ref.

If `items` has not changed since the initial load `sumA` is more efficient as it leverages caching. The array reduction only happens once. `total` is equally efficient for reading its stored value, but the overall approach in Option A is preferred for derived state because it manages the calculation lifecycle automatically and optimally.

A clear example where the watch (Option B) approach is necessary is when the sum calculation needs to trigger a server request:

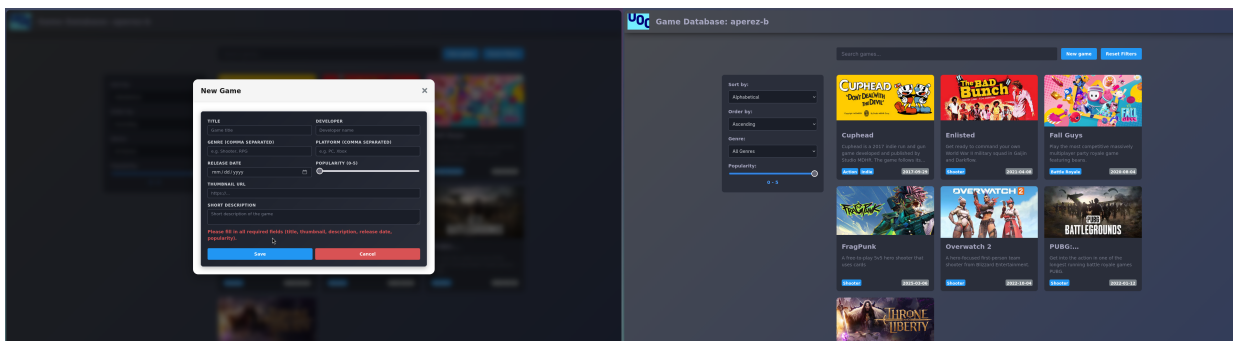
```
watch(total, (newTotal) => {
  if (newTotal > 100) {
    // This is a side effect and must be done in a watcher
    alert('Total exceeded 100! Sending notification to server...');
    // await axios.post('/api/total-exceeded', { newTotal })
  }
});
```

Project

Note

The code for this assignment can be found [here](#).

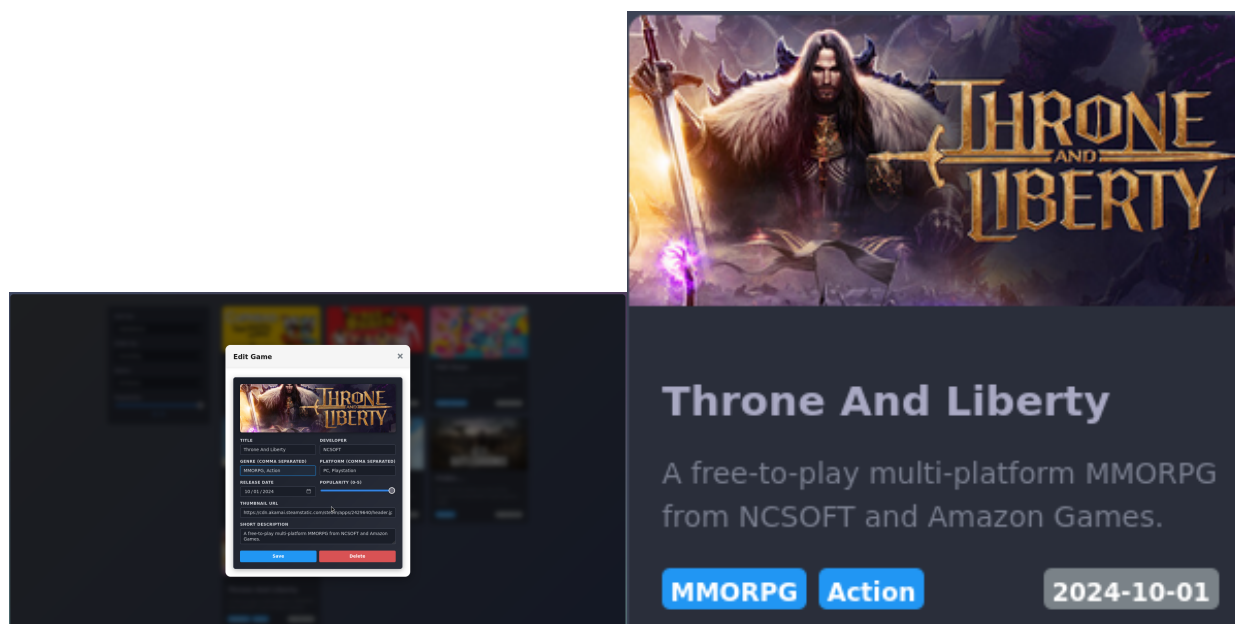
Here are some screenshots showing some of the requested features in action:



(a) Show an error if required parameters are not specified

(b) Successful game creation

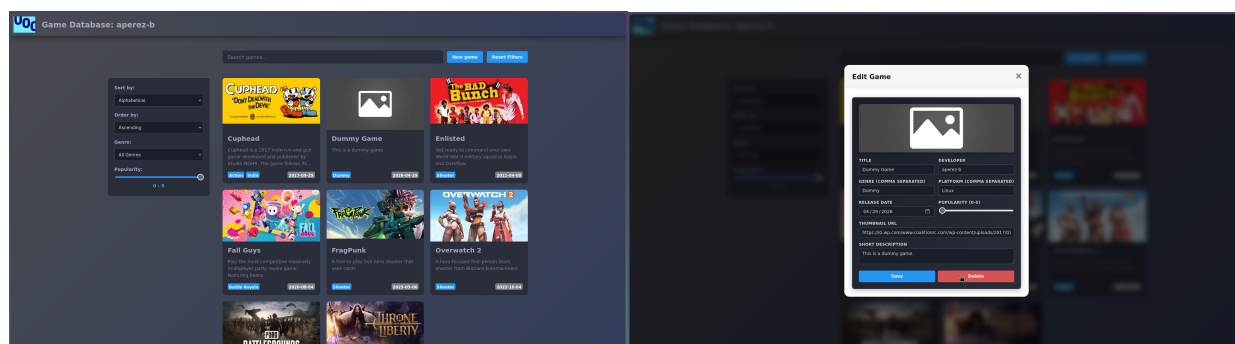
Figure 1: Process of adding the game Cuphead to the app



(a) Add "Action" genre to game

(b) Verify the new genre is applied

Figure 2: Process of adding the "Action" genre to the game "Throne and Liberty"



(a) Create a dummy game to be deleted

(b) Delete menu for dummy game

Figure 3: Process of deleting a sample game from the library

References

Azaustre Rodriguez, Carlos. 2024. *Advanced Concepts and Techniques of Web Programming*. 1st ed. Fundació Universitat Oberta de Catalunya (FUOC).

LogRocket Blog. 2020. “Positioning Elements with Vue 3 Teleport.” LogRocket. <https://blog.logrocket.com/positioning-elements-with-vue-3-teleport/>.

Vue.js. 2024a. “Computed Properties.” Vue.js. <https://vuejs.org/guide/essentials/computed.html>.

Vue.js. 2024b. “Lifecycle Hooks.” Vue.js. <https://vuejs.org/guide/essentials/lifecycle.html>.

Vue.js. 2024c. “Props.” Vue.js. <https://vuejs.org/guide/components/props>.